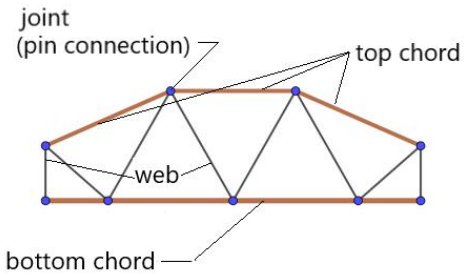
STATICS - METHOD OF NODES

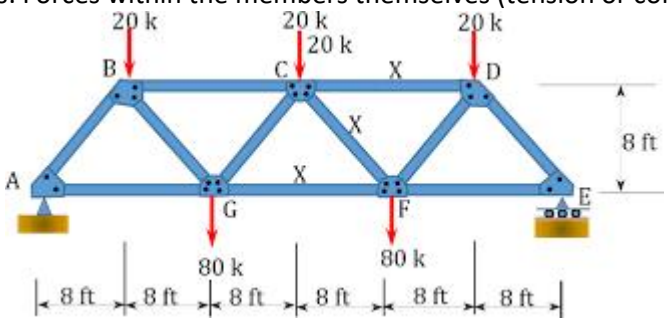
The Method of Nodes is a technique used to determine the forces acting on members within a truss structure. Trusses are structures made of interconnected straight members that form a network of triangles. These are commonly used in bridges, roofs, and other structures.

Key Concepts

- Joints (Nodes): The points where truss members connect.
- Members: The individual straight pieces of the truss.



- Forces: Forces can be either:
  External Loads: Forces applied to the truss from the outside (e.g., weight, wind).
  Internal Forces: Forces within the members themselves (tension or compression).



- Equilibrium: For a truss to be stable, every joint must be in equilibrium (the sum of forces in both the x and y directions must be zero).

Steps in the Method of Nodes

1. Free Body Diagram (FBD)

i. Isolate a single joint.

ii. Represent each member connected to that joint with a force vector acting along the member's axis.

iii. Assume the direction of the force (tension or compression). If your assumption is wrong, the math will give you a negative result.

iv. Include any external forces acting on the joint.

2. Equilibrium Equations

i. Apply the equations of equilibrium to the joint:

$\Sigma F_x = 0$ (sum of forces in the x-direction equals zero)

$\Sigma F_y = 0$ (sum of forces in the y-direction equals zero)

3. Solve

i. Solve the equations to find the magnitudes and correct directions of the unknown forces in the members

4. Repeat

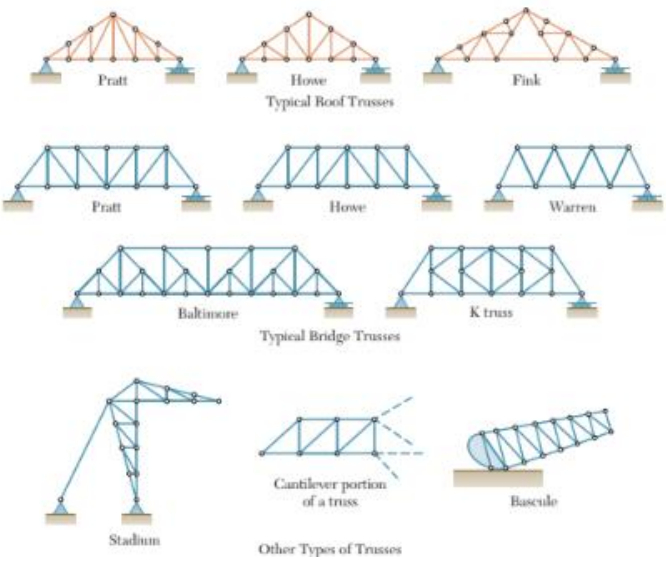
i. Move to another joint and repeat the process until you've found the forces in all the members.

Important Considerations

- Start with a Joint: Begin at a joint with no more than two unknown forces. This is because you have only two equilibrium equations at each joint.

Tension vs. Compression:

- Tension: A member is being pulled apart (force acts away from the joint).
- Compression: A member is being pushed together (force acts towards the joint).



CODE FOR TRUSS ANALYSIS

```
% 2D TRUSS ANALYSIS
% STATICS

%% THANASIS CHARALAMBOUS
% Mechanical Engineer
% 2025/01

%% CODE
clear
clc

%% INSTRUCTIONS
% SET the values of PART 1 & PART 2, PART 3A accordingly to your problem
% No need to change anything beyond PART 3A
% The supports should be at x=0 to make the code works

%% PART 1: GEOMETRY OF TRUSS

% SET the geometry of Truss
% Nodes of Coordinates (x,y)
% They can as many nodes as it needed
nodes = [
   0 6.4;    % Node 1 coordinates (x, y)
   12 6.4;    % Node 2 coordinates
   24 6.4;   % Node 3 coordinates
   0 0;   % Node 4 coordinates
   12 0;   % Node 5 coordinates
   24  0;   % Node 6 coordinates
   ];
num_nodes = size(nodes, 1); % Number of nodes

% SET Members of the truss
% Which nodes are connect between them by the members
members = [
   1 2; % Member 1 connects nodes 1 and 2
   1 4; % Member 2
   2 3; % Member 3
   2 4; % Member 4
   2 5; % Member 5
   2 6; % Member 6 connects nodes 2 and 6
   3 6; % Member 7 connects nodes 3 and 6
   4 5; % Member 8
   5 6; % Member 9
   ];
num_members = size(members,1); % Number of members

%% PART 2: EXTERNAL FORCES & MOMENTS
% SET External loads & moments [Rx, Ry, M] = [Horizontal force, vertical, Moment]
% If there is load with angle, set it with its x,y components

loads = [
   0 -1 0 ;  % Node 1: No horizontal (0),vertical -1 units load, no moment
   0 -4 0 ;     % Node 2:
```

```
   0 -1 0 ;     % Node 3
   0 0 0 ;     % Node 4: No loads, no moment
   0 -2.4 0 ;  % Node 5
   0 0 0 ];  % Node 6: No Horizontal load, no vertical, no moment

%% PART 3: SUPPORTS or CONNECTIONS (Applying Boundary Conditions)
% Define support conditions
% What movements each type of support restricts.

% Define support types at each node
% 0: No support
% 1: Roller, Rocker, Frictionless surface ON GROUND
% 2: Roller, Rocker, Frictionless surface ON WALL
% 3: Short Cable, Short Link
% 4: Collar on frictionless rod, Frictionless pin in slot
% 5: Hinge or frictionless pin or rough surface
% 6: Fixed support

% PART 3A
% SET support types for each node
% 0, 1, 2, 3, 4 or 5
support_node = [
   0; % Roller at node 1
   0; % No support at node 2
   0; % No support at node 3
   1; % No support at node 4
   0; % Hinge at node 5
   4]; % No support at node 6

% PART 3B
% !!! DO NOT CHANGE THESE !!!
% Support Types, Reactions types : [Rx, Ry, M]
support_types = [
   0 1 0 ;  % Support Type 1: Roller, Rocker, Frictionless Surface (only vertical reaction)
   0 0 0 ;  % Support Type 2: Short Cable, Short Link (no reactions defined yet)
   0 0 0 ;  % Support Type 3: Collar on Frictionless Rod, Frictionless Pin in Slot (needs further
calculation for reactions)
   1 1 0 ;  % Support Type 4: Hinge, Frictionless Pin, Rough Surface (horizontal & vertical reactions, no
moments)
   1 1 1 ;  % Support Type 5: Fixed Support (horizontal & vertical reactions, moments)
];


%% PART 4: Plotting the Geometry of Truss
% Figure 1
figure(1)
hold on;
axis equal;
grid on;
xlabel('x coordinates (m)');
ylabel('y coordinates (m)');
title('Truss Network, Nodes & Members numbering ');

% Plot Nodes
for i = 1:length(nodes) % or i = 1:size(nodes,1);
   plot(nodes(i,1) , nodes(i,2), 'ko', 'MarkerFaceColor','k', 'LineWidth', 1.5); %% Plot node as a black
circle
```

```
   text(nodes(i,1) + 0.5, nodes(i,2) - 0.5, ['N' num2str(i)], 'Fontsize', 12, 'Color', 'k'); % Label the
node(N1-N..)
end

% Plot Members
for i = 1:length(members) % or i = 1:size(members,1);
   node1 = members(i,1); % 1st node of the member
   node2 = members(i,2); % 2nd node of the member
   % Plot Members, line from 1 node to other node
   plot( [nodes(node1,1), nodes(node2,1)], [ nodes(node1,2), nodes(node2,2) ], 'b-', 'LineWidth', 2);
   % Label members
   midpoint = [(nodes(node1,1) + nodes(node2,1))/2, (nodes(node1,2) + nodes(node2,2))/ 2 ];
   text(midpoint(1)  , midpoint(2), ['M' num2str(i)], 'FontSize', 12 , 'Color', 'b') % Label the
members(M1-M..)
end
hold off;

%% PART 5: Joint connectivity (members that are connected to each node)
joint_connectivity = cell(num_nodes,1); % Create an empty cell array to hold the joint connectivity
   % Loop through each member to check which nodes are connected between which members
   for i = 1:num_members
      node1 = members(i, 1); % 1st node of the member
      node2 = members(i, 2); % 2nd node of the member
      % Add the member index to connectivity list for both nodes
      joint_connectivity{node1} = unique([joint_connectivity{node1},i]); % unique ensure that the
member index is not shown multiple times for each node
      joint_connectivity{node2} = unique([joint_connectivity{node2},i]);
   end

%% PART 6: Nodes' Support Reactions
% Assign the Nodes' support reactions

% Initialize an empty vector to store reaction forces & moments
reaction_forces_moments = zeros(length(nodes),3);
    % Loop through each node and assign its support reaction
    for i = 1:length(nodes)
       support_type = support_node(i); % Get the support type for node i
      if support_type == 1
        reaction_forces_moments(i,:) = support_types(1,:);
      elseif support_type == 2
        reaction_forces_moments(i,:) = support_types(2,:);
      elseif support_type == 3
        reaction_forces_moments(i,:) = support_types(3,:);
      elseif support_type == 4
        reaction_forces_moments(i,:) = support_types(4,:);
      elseif support_type == 5
        reaction_forces_moments(i,:) = support_types(5,:);
      else
        reaction_forces_moments(i, :) = [0 0 0]; % No support at this node (free)
      end
    end

%% PART 7: Plot External Loads

figure(2);
hold on;
axis equal;
```

```
grid on;
xlabel('x coordinates (m)');
ylabel('y coordinates (m)');
title('External Loads on Truss');

% Plot Nodes
for i = 1:length(nodes) % or i = 1:size(nodes,1);
   plot(nodes(i,1) , nodes(i,2), 'o','LineWidth', 1.5); %% Plot node as a black circle
end

% Plot Members
for i = 1:length(members) % or i = 1:size(members,1);
   node1 = members(i,1); % 1st node of the member
   node2 = members(i,2); % 2nd node of the member
   % Plot Members, line from 1 node to other node
   plot( [nodes(node1,1), nodes(node2,1)], [ nodes(node1,2), nodes(node2,2) ], 'LineWidth', 3,
'Color','k');
end

% Plot External Loads (Forces)
for i = 1:length(loads)
   Fx = loads(i,1) / abs(min(min(loads))/10 ); % Horizontal load at node i
   Fy = loads(i,2) / abs(min(min(loads))/10 ); % Vertical load at node i
   % Only plot non-zero forces
   if Fx ~= 0
     quiver(nodes(i,1), nodes(i,2), Fx, 0, 'r', 'LineWidth', 2,'MaxHeadSize', 3.0);
     text(nodes(i,1) , nodes(i,2) , sprintf('%.1f N', loads(i,1)), 'Color', 'r', 'FontSize', 10);
   end
   if Fy ~= 0
     quiver(nodes(i, 1), nodes(i, 2), 0, Fy, 'r', 'LineWidth', 2, 'MaxHeadSize', 3.0); % +
abs(min(min(loads(i,2)))/60) That is adjustment of Force arrows position
     text(nodes(i,1)  , nodes(i,2)  , sprintf('%.1f N', loads(i,2)), 'Color', 'r', 'FontSize', 10);
   end
end
hold off;

%% PART 8: ANALYSIS
% Network Equilibrium
% Sum of Forces (SFx=0, SFy=0)

% Creation of Reactions symbolic matrix (sym_reactions)
[rows, cols] = size(reaction_forces_moments);
reactions = strings(rows,cols); % Create reactions string array
base_names = ["Rx", "Ry", "M"]; % Names corresponding to columns
% Naming Supports Reactions
for i = 1:rows
   for j = 1:cols
      if reaction_forces_moments (i,j) == 1
        reactions(i,j) = base_names(j) + num2str(i); % Assign symbolic variable
      else
        reactions(i,j) = 0 ;
      end
   end
end
% Initialize a cell array
sym_reactions = cell(size(reactions));
% Create Sym Reactions
```

```
for i = 1:numel(reactions) % numel, total number of arrays
   if ~strcmp(reactions{i}, "0") % if reactions{i} == 0
      sym_reactions{i} = sym(reactions{i}); % Replace with symbolic variable
   else
      sym_reactions{i} = 0; % Assign numeric zero for known values
   end
end
% Convert to symbolic matrix
sym_reactions = cell2sym(sym_reactions);

% Define External Loads
external_loads = loads;
node_distances_x = nodes(:, 1); % Horizontal positions of nodes (x-coordinates)
node_distances_y = nodes(:, 2); % Vertical positions of nodes (y-coordinates)

% Initialize equilibrium equations
eq_SFx = 0; % Sum of forces in X
eq_SFy = 0; % Sum of forces in Y
eq_SM = 0;  % Sum of moments

% Loop through each node to calculate contributions to equilibrium
for  i = 1: rows
    % Add contributions to SFx and SFy
    eq_SFx = eq_SFx + sym_reactions(i,1) + external_loads(i,1);
    eq_SFy = eq_SFy + sym_reactions(i,2) + external_loads(i,2);

    % Add contributions to SM
    eq_SM = sum((sym_reactions(:,2) + external_loads(:,2)) .* node_distances_x - (sym_reactions(:,1) +
external_loads(:,1)) .* node_distances_y);
end

% Solve the equilibrium equations for the unknown reactions
unknowns = symvar(sym_reactions); % Extract all symbolic variables (unknowns), symvar finds the
unknowns variables
solutions = solve([eq_SFx == 0, eq_SFy == 0, eq_SM == 0], unknowns); % Solve equations to find
unknowns

% Display the solutions for the reactions
disp('Supports Reaction Forces and Moments');
disp(solutions);

% Get all field names from the struct
fields = fieldnames(solutions); % Return the field names in a cell array using the fieldnames
% Loop through each field and update the matrix
for i = 1:numel(fields)
   field = fields{i};  % Get the current field name (e.g., 'Rx5')
   value = solutions.(field);% Get the corresponding value
   % Extract the row number from the field name
   var_type = field(2); % 'x' or 'y' or another variable
   row_num = str2double(field(3:end)); % Convert strings to double precision values (numbers of the
field(3rd and after))

% Determine the column index based on the type (customizable)
   switch var_type
      case 'x'
         col_num = 1;  % X forces/moments go to column 1
      case 'y'
```

```
        col_num = 2;  % Y forces/moments go to column 2
      otherwise
        error('Unknown reaction force/moment type: %s', var_type);
    end
% Update the matrix
reaction_forces_moments(row_num, col_num) = value;
end
% Display the updated matrix
disp('Reaction Forces and Moments at Each Node (Rx, Ry, M):');
disp(reaction_forces_moments);

%% PART 10: Method of Nodes
% Solve for Member Forces with Known Support Reactions

% Initialize symbolic forces in members
member_forces = sym('F', [num_members,1]); % creates an array(num_member*1)
reaction_forces_x = sym('Rx',[num_members,1]);
reaction_forces_y = sym('Ry',[num_members,1]);

% Get Reaction forces
known_reactions = []; % array creation
known_values = [];
for i = 1:num_nodes
    for j =1:2 % Check Rx (column 1) and Ry (column 2)
      reaction_value = reaction_forces_moments(i,j); % Get from input matrix
      if reaction_value ~= 0
        if j == 1 % Rx direction
          known_reactions = [known_reactions; reaction_forces_x(i)]; % Store reaction force
        else % Ry direction
          known_reactions = [known_reactions; reaction_forces_y(i)]; % Store reaction force
        end
         known_values = [known_values; reaction_value]; % Store reaction values
      end
    end
end

% Initialize Equilibrium equations
eqs = [];
for i = 1:num_nodes
    % Initialize sum of forces in x and y directions
    Fx_eq = sym(0);
    Fy_eq = sym(0);
    % External forces at the node
    Fx_ext = external_loads(i,1);
    Fy_ext = external_loads(i,2);
    % Reaction forces at the node (if unknown, they will remain symbolic)
    Rx = reaction_forces_x(i);  % Rx for the node i
    Ry = reaction_forces_y(i);  % Ry for the node i
    % Add reaction forces to equilibrium
    Fx_eq = Fx_eq + Rx;
    Fy_eq = Fy_eq + Ry;

    % Members connected to the current node
    connected_members = joint_connectivity{i};
    for j = 1:length(connected_members)
      member_index = connected_members(j);
      member_nodes = members(member_index, :); % Nodes connected by this member
```

```
        other_node = setdiff(member_nodes, i);  % Find the node that's not i

        % Compute directional cosines (unit vector components)
        dx = nodes(other_node,1) -  nodes(i, 1); % x distance between the 2 nodes
        dy = nodes(other_node,2) -  nodes(i, 2); % y distance between the 2 nodes
        L = sqrt(dx.^2 + dy.^2);  % Length of the member

        % Ensure no zero-length members are present
        if L == 0
            error('Error: Zero-length member detected!');
        end

        % Unit vector components
        cos_theta = dx / L;
        sin_theta = dy / L;

        % Add contributions to equilibrium equations from member forces
        Fx_eq = Fx_eq + member_forces(member_index) * cos_theta;
        Fy_eq = Fy_eq + member_forces(member_index) * sin_theta;
    end

    % Add external forces at the node
    Fx_eq = Fx_eq + Fx_ext;
    Fy_eq = Fy_eq + Fy_ext;

% Store the equilibrium equations
    eqs = [eqs; Fx_eq == 0; Fy_eq == 0];

end

% Print Equilibrium Equations Before Solving
disp('Equilibrium Equations:');
for i = 1:length(eqs)
    fprintf('Equation %d: %s\n',i,char(eqs(i)));
end

% Substitute known Support Reactions
if ~isempty(known_reactions)
     eqs = subs(eqs, known_reactions, known_values);
end

% Solve system of equations for member forces
unknown_forces = [member_forces; reaction_forces_x(:); reaction_forces_y(:)];
solution = solve(eqs, unknown_forces);
% Check if the solution is empty or invalid
if isempty(solution)
    error('Error: The system of equations has no solution.');
end

% Extract numeric values from solution structure
member_forces_values = zeros(num_members, 1);  % Initialize storage
reaction_forces_values = zeros(num_nodes, 2);  % Store solved reaction forces

% Extract solved member forces
for i = 1:num_members
    force_var = ['F', num2str(i)];
    if isfield(solution, force_var) && ~isempty(solution.(force_var))
```

```
        member_forces_values(i) = double(solution.(force_var));
    else
        member_forces_values(i) = NaN; % Indicate unsolved force
        fprintf('Warning: Member %d force could not be determined.\n', i);
    end
end

% Print the results for member forces
disp('Solved Member Forces:');
disp('* Positive(+): Tensile Force');
disp('** Negative(-): Compressive Force');
for i = 1:num_members
    member_nodes = members(i,:);  % Get the nodes for this member
    node_pair_str = sprintf('F%d%d', member_nodes(1), member_nodes(2)); % Create F_ij string

    fprintf('F%d = %s = %.2f\n', i,node_pair_str, member_forces_values(i));
end

% Extract solved reaction forces
for i = 1:num_nodes
    % Rx for node i
    if isfield(solution, ['Rx', num2str(i)]) && ~isempty(solution.(['Rx', num2str(i)]))
        reaction_forces_values(i, 1) = double(solution.(['Rx', num2str(i)]));
    else
        reaction_forces_values(i, 1) = 0; % Default to zero if not solved
    end

    % Ry for node i
    if isfield(solution, ['Ry', num2str(i)]) && ~isempty(solution.(['Ry', num2str(i)]))
        reaction_forces_values(i, 2) = double(solution.(['Ry', num2str(i)]));
    else
        reaction_forces_values(i, 2) = 0; % Default to zero if not solved
    end
end
```

PROBLEMS SOLVE BY CODE

(Vector Mechanics for Engineers Statics and Dynamics, 9th Edition by Ferdinand P. Beer, E. Russell Johnston, Jr., David F. Mazurek, Phillip J. Cornwell, Elliot R. Eisenberg)

1)

6.1 through 6.8 Using the method of joints, determine the force in each member of the truss shown. State whether each member is in tension or compression.

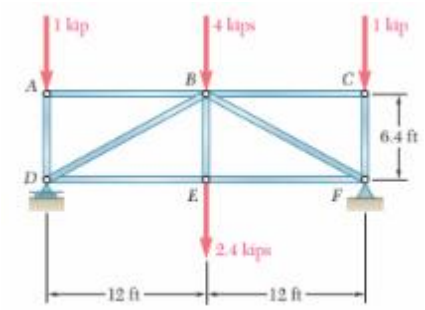
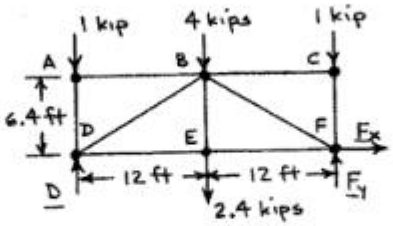


Fig. P6.4

SOLUTION

Reactions:



$$+\circlearrowleft \Sigma M_D = 0: \quad F_y(24) - (4+2.4)(12) - (1)(24) = 0$$

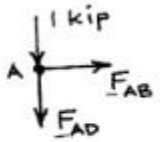
$$\mathbf{F}_y = 4.2 \text{ kips}\uparrow$$

$$\Sigma F_x = 0: \quad \mathbf{F}_x = 0$$

$$+\uparrow \Sigma F_y = 0: \quad D - (1+4+1+2.4) + 4.2 = 0$$

$$\mathbf{D} = 4.2 \text{ kips}\uparrow$$

Joint A:

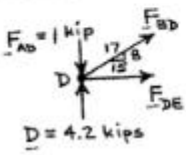


$$\Sigma F_x = 0: \quad F_{AB} = 0 \qquad F_{AB} = 0 \blacktriangleleft$$

$$+\uparrow \Sigma F_y = 0: \quad -1 - F_{AD} = 0$$

$$F_{AD} = -1 \text{ kip} \qquad F_{AD} = 1.000 \text{ kip} \quad C \blacktriangleleft$$

Joint D:



$$+\uparrow \Sigma F_y = 0: \quad -1 + 4.2 + \frac{8}{17} F_{BD} = 0$$

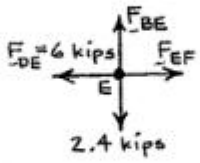
$$F_{BD} = -6.8 \text{ kips} \qquad F_{BD} = 6.80 \text{ kips} \quad C \blacktriangleleft$$

$$\xrightarrow{+} \Sigma F_x = 0: \quad \frac{15}{17}(-6.8) + F_{DE} = 0$$

$$F_{DE} = +6 \text{ kips} \qquad F_{DE} = 6.00 \text{ kips} \quad T \blacktriangleleft$$

PROBLEM 6.4 (Continued)

Joint E:

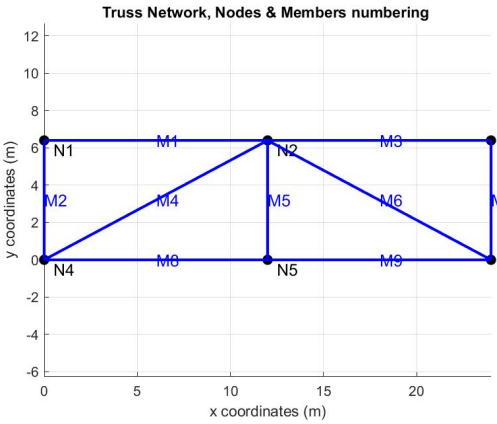


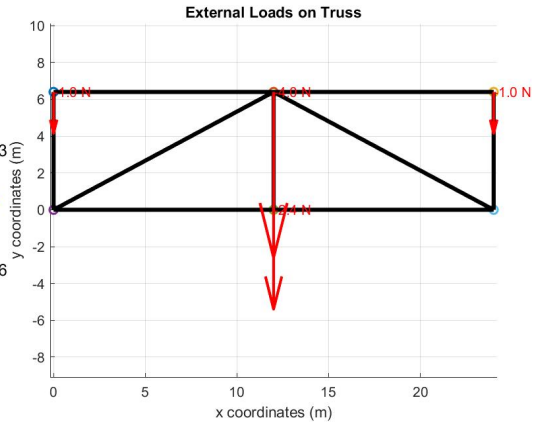
$+\uparrow \Sigma F_y = 0: \quad F_{BE} - 2.4 = 0$

$F_{BE} = +2.4$ kips $\qquad F_{BE} = 2.40$ kips T ◀

Truss and loading symmetrical about ℄

Code Solution:





Supports Reaction Forces and Moments

Rx6: 0
Ry4: 4.2
Ry6: 4.2

Equilibrium Equations:

Equation 1: F1 + Rx1 == 0
Equation 2: Ry1 - F2 - 1 == 0
Equation 3: F3 - F1 - (15*F4)/17 + (15*F6)/17 + Rx2 == 0
Equation 4: Ry2 - F5 - (8*F6)/17 - (8*F4)/17 - 4 == 0
Equation 5: Rx3 - F3 == 0
Equation 6: Ry3 - F7 - 1 == 0
Equation 7: (15*F4)/17 + F8 + Rx4 == 0
Equation 8: F2 + (8*F4)/17 + Ry4 == 0
Equation 9: F9 - F8 + Rx5 == 0
Equation 10: F5 + Ry5 - 12/5 == 0
Equation 11: Rx6 - F9 - (15*F6)/17 == 0
Equation 12: (8*F6)/17 + F7 + Ry6 == 0

Solved Member Forces:

* Positive(+): Tensile Force
** Negative(-): Compressive Force
F1 = F12 = 0.00
F2 = F14 = -1.00
F3 = F23 = 0.00
F4 = F24 = -6.80

F5 = F25 = 2.40
F6 = F26 = -6.80
F7 = F36 = -1.00
F8 = F45 = 6.00
F9 = F56 = 6.00

2)

6.11 Determine the force in each member of the Howe roof truss shown. State whether each member is in tension or compression.

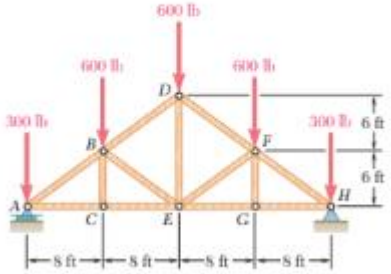
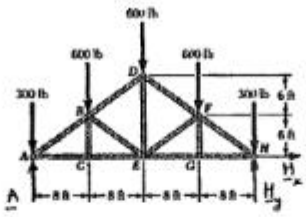


Fig. P6.11

SOLUTION

Free body: Truss



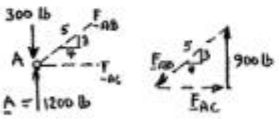
$$\Sigma F_x = 0: \quad \mathbf{H}_x = 0$$

Because of the symmetry of the truss and loading:

$$A = H_y = \frac{1}{2} \text{ Total load}$$

$$\mathbf{A} = \mathbf{H}_y = 1200 \text{ lb}\uparrow$$

Free body: Joint A:



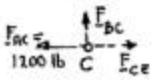
$$\frac{F_{AB}}{5} = \frac{F_{AC}}{4} = \frac{900 \text{ lb}}{3}$$

$\mathbf{F}_{AB} = 1500 \text{ lb} \quad C$ ◀

$\mathbf{F}_{AC} = 1200 \text{ lb} \quad T$ ◀

Free body: Joint C:

BC is a zero-force member



$\mathbf{F}_{BC} = 0$

$\mathbf{F}_{CE} = 1200 \text{ lb} \quad T$ ◀

PROBLEM 6.11 (Continued)

Free body: Joint B:

$$\xrightarrow{+} \Sigma F_x = 0: \quad \frac{4}{5}F_{BD} + \frac{4}{5}F_{BE} + \frac{4}{5}(1500 \text{ lb}) = 0$$

or $$F_{BD} + F_{BE} = -1500 \text{ lb} \qquad (1)$$

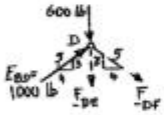
$$+\uparrow \Sigma F_y = 0: \quad \frac{3}{5}F_{BD} - \frac{3}{5}F_{BE} + \frac{3}{5}(1500 \text{ lb}) - 600 \text{ lb} = 0$$

or $$F_{BD} - F_{BE} = -500 \text{ lb} \qquad (2)$$

Add Eqs. (1) and (2): $2F_{BD} = -2000 \text{ lb}$ $\qquad F_{BD} = 1000 \text{ lb} \quad C$ ◀

Subtract (2) from (1): $2F_{BE} = -1000 \text{ lb}$ $\qquad F_{BE} = 500 \text{ lb} \quad C$ ◀

Free Body: Joint D:



$$\xrightarrow{+} \Sigma F_x = 0: \quad \frac{4}{5}(1000 \text{ lb}) + \frac{4}{5}F_{DF} = 0$$

$F_{DF} = -1000 \text{ lb}$ $\qquad F_{DF} = 1000 \text{ lb} \quad C$ ◀

$$+\uparrow \Sigma F_y = 0: \quad \frac{3}{5}(1000 \text{ lb}) - \frac{3}{5}(-1000 \text{ lb}) - 600 \text{ lb} - F_{DE} = 0$$

$F_{DE} = +600 \text{ lb}$ $\qquad F_{DE} = 600 \text{ lb} \quad T$ ◀

Because of the symmetry of the truss and loading, we deduce that

$F_{EF} = F_{BE}$ $\qquad F_{EF} = 500 \text{ lb} \quad C$ ◀

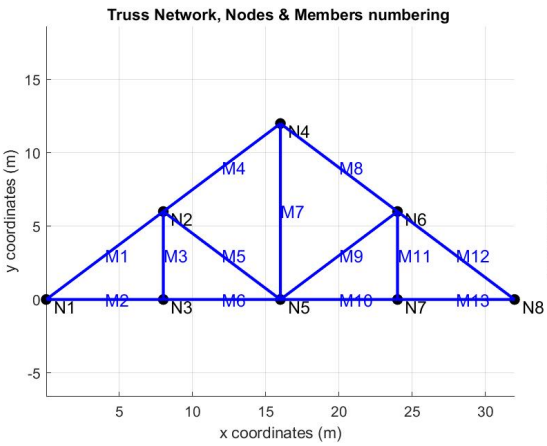
$F_{EG} = F_{CE}$ $\qquad F_{EG} = 1200 \text{ lb} \quad T$ ◀

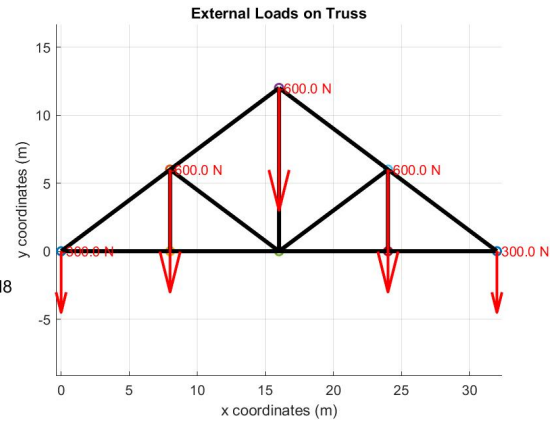
$F_{FG} = F_{BC}$ $\qquad F_{FG} = 0$ ◀

$F_{FH} = F_{AB}$ $\qquad F_{FH} = 1500 \text{ lb} \quad C$ ◀

$F_{GH} = F_{AC}$ $\qquad F_{GH} = 1200 \text{ lb} \quad T$ ◀

Code Solution:





Supports Reaction Forces and Moments

Rx8: 0
Ry1: 1200
Ry8: 1200

Equilibrium Equations:

Equation 1: (4*F1)/5 + F2 + Rx1 == 0
Equation 2: (3*F1)/5 + Ry1 - 300 == 0
Equation 3: (4*F4)/5 - (4*F1)/5 + (4*F5)/5 + Rx2 == 0
Equation 4: (3*F4)/5 - F3 - (3*F1)/5 - (3*F5)/5 + Ry2 - 600 == 0
Equation 5: F6 - F2 + Rx3 == 0

Equation 6: F3 + Ry3 == 0
Equation 7: (4*F8)/5 - (4*F4)/5 + Rx4 == 0
Equation 8: Ry4 - F7 - (3*F8)/5 - (3*F4)/5 - 600 == 0
Equation 9: (4*F9)/5 - F6 - (4*F5)/5 + F10 + Rx5 == 0
Equation 10: (3*F5)/5 + F7 + (3*F9)/5 + Ry5 == 0
Equation 11: (4*F12)/5 - (4*F9)/5 - (4*F8)/5 + Rx6 == 0
Equation 12: (3*F8)/5 - (3*F9)/5 - F11 - (3*F12)/5 + Ry6 - 600 == 0
Equation 13: F13 - F10 + Rx7 == 0
Equation 14: F11 + Ry7 == 0
Equation 15: Rx8 - F13 - (4*F12)/5 == 0
Equation 16: (3*F12)/5 + Ry8 - 300 == 0

Solved Member Forces:
* Positive(+): Tensile Force
** Negative(-): Compressive Force
F1 = F12 = -1500.00
F2 = F13 = 1200.00
F3 = F23 = 0.00
F4 = F24 = -1000.00
F5 = F25 = -500.00
F6 = F35 = 1200.00
F7 = F45 = 600.00
F8 = F46 = -1000.00
F9 = F56 = -500.00
F10 = F57 = 1200.00
F11 = F67 = 0.00
F12 = F68 = -1500.00
F13 = F78 = 1200.00

3)

6.13 Determine the force in each member of the truss shown.

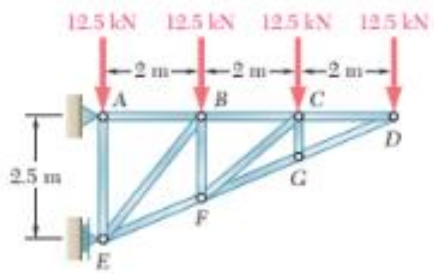
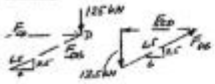


Fig. P6.13

SOLUTION

Joint D:

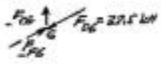


$$\frac{12.5\text{ kN}}{2.5} = \frac{F_{CD}}{6} = \frac{F_{DG}}{6.5}$$

$F_{CD} = 30\text{ kN}\quad T$ ◀

$F_{DG} = 32.5\text{ kN}\quad C$ ◀

Joint G:

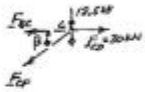


$\nwarrow \Sigma F = 0:\quad F_{CG} = 0$ ◀

$\nearrow \Sigma F = 0:\quad F_{FG} = 32.5\text{ kN}\quad C$ ◀

Joint C:

$$BF = \frac{2}{3}(2.5\text{ m}) = 1.6667\text{ m}\qquad \beta = \angle BCF = \tan^{-1}\frac{BF}{2} = 39.81°$$



$$+\uparrow \Sigma F_y = 0:\quad -12.5\text{ kN} - F_{CF}\sin\beta = 0$$

$$-12.5\text{ kN} - F_{CF}\sin 39.81° = 0$$

$$F_{CF} = -19.526\text{ kN}$$

$F_{CF} = 19.53\text{ kN}\quad C$ ◀

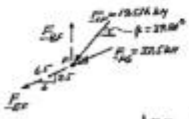
$$\xrightarrow{+} \Sigma F_x = 0:\quad 30\text{ kN} - F_{BC} - F_{CF}\cos\beta = 0$$

$$30\text{ kN} - F_{BC} - (-19.526\text{ kN})\cos 39.81° = 0$$

$$F_{BC} = +45.0\text{ kN}$$

$F_{BC} = 45.0\text{ kN}\quad T$ ◀

Joint F:



$$\xrightarrow{+} \Sigma F_x = 0:\quad -\frac{6}{6.5}F_{EF} - \frac{6}{6.5}(32.5\text{ kN}) - F_{CF}\cos\beta = 0$$

$$F_{EF} = -32.5\text{ kN} - \left(\frac{6.5}{6}\right)(19.526\text{ kN})\cos 39.81°$$

$$F_{EF} = -48.75\text{ kN}$$

$F_{EF} = 48.8\text{ kN}\quad C$ ◀

$$+\uparrow \Sigma F_y = 0:\quad F_{BF} - \frac{2.5}{6.5}F_{EF} - \frac{2.5}{6.5}(32.5\text{ kN}) - (19.526\text{ kN})\sin 39.81° = 0$$

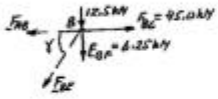
$$F_{BF} - \frac{2.5}{6.5}(-48.75\text{ kN}) - 12.5\text{ kN} - 12.5\text{ kN} = 0$$

$$F_{BF} = +6.25\text{ kN}$$

$F_{BF} = 6.25\text{ kN}\quad T$ ◀

PROBLEM 6.13 (Continued)

Joint B:



$$\tan\alpha = \frac{2.5\text{ m}}{2\text{ m}};\quad \gamma = 51.34°$$

$$+\uparrow \Sigma F_y = 0:\quad -12.5\text{ kN} - 6.25\text{ kN} - F_{BE}\sin 51.34° = 0$$

$$F_{BE} = -24.0\text{ kN}$$

$F_{BE} = 24.0\text{ kN}\quad C$ ◀

$$\xrightarrow{+} \Sigma F_x = 0:\quad 45.0\text{ kN} - F_{AB} + (24.0\text{ kN})\cos 51.34° = 0$$

$$F_{AB} = +60\text{ kN}$$

$F_{AB} = 60.0\text{ kN}\quad T$ ◀

Joint E:

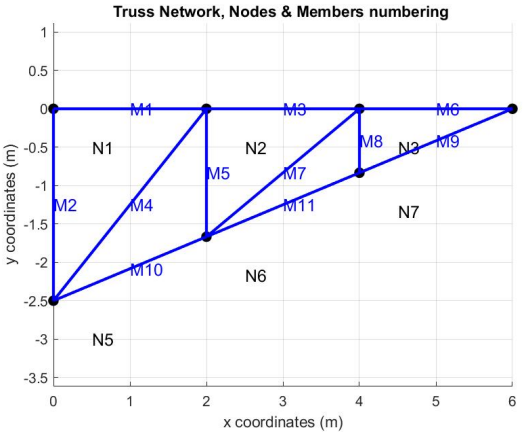
$$\gamma = 51.34°$$

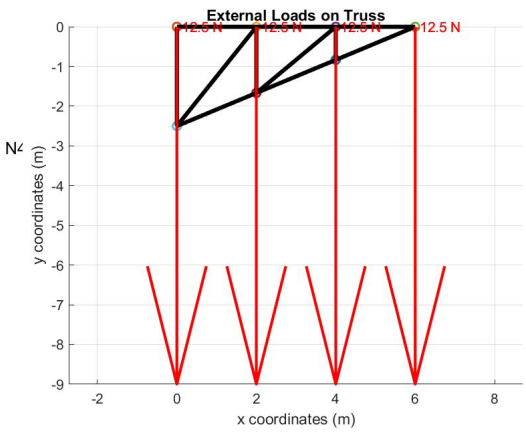
$$\xrightarrow{+} \Sigma F_y = 0:\quad F_{AE} - (24\text{ kN})\sin 51.34° - (48.75\text{ kN})\frac{2.5}{6.5} = 0$$

$$F_{AE} = +37.5\text{ kN}$$

$F_{AE} = 37.5\text{ kN}\quad T$ ◀

CODE SOLUTION:





Supports Reaction Forces and Moments

Rx1: -60

Rx5: 60
Ry1: 50

Equilibrium Equations:
Equation 1: F1 + Rx1 == 0
Equation 2: Ry1 - F2 - 25/2 == 0
Equation 3: F3 - F1 + Rx2 - (4*41^(1/2)*F4)/41 == 0
Equation 4: Ry2 - F5 - (5*41^(1/2)*F4)/41 - 25/2 == 0
Equation 5: F6 - F3 + Rx3 - (6*61^(1/2)*F7)/61 == 0
Equation 6: Ry3 - F8 - (5*61^(1/2)*F7)/61 - 25/2 == 0
Equation 7: Rx4 - (12*F9)/13 - F6 == 0
Equation 8: Ry4 - (5*F9)/13 - 25/2 == 0
Equation 9: (12*F10)/13 + Rx5 + (4*41^(1/2)*F4)/41 == 0
Equation 10: F2 + (5*F10)/13 + Ry5 + (5*41^(1/2)*F4)/41 == 0
Equation 11: (12*F11)/13 - (12*F10)/13 + Rx6 + (6*61^(1/2)*F7)/61 == 0
Equation 12: F5 - (5*F10)/13 + (5*F11)/13 + Ry6 + (5*61^(1/2)*F7)/61 == 0
Equation 13: (12*F9)/13 - (12*F11)/13 + Rx7 == 0
Equation 14: F8 + (5*F9)/13 - (5*F11)/13 + Ry7 == 0

Solved Member Forces:
* Positive(+): Tensile Force
** Negative(-): Compressive Force
F1 = F12 = 60.00
F2 = F15 = 37.50
F3 = F23 = 45.00
F4 = F25 = -24.01
F5 = F26 = 6.25
F6 = F34 = 30.00
F7 = F36 = -19.53
F8 = F37 = 0.00
F9 = F47 = -32.50
F10 = F56 = -48.75
F11 = F67 = -32.50